

UNIVERSIDADE
ESTRUTURAS DE DADOS II

ÁRVORES RUBRO-NEGRAS

(Red-Black Trees)

Propriedades, Operações, Implementação e Análise de Complexidade

Trabalho de Seminário apresentado à disciplina de
Estruturas de Dados II
sob orientação do Professor Tiago Lima

EQUIPE 3

Adilson · Chris · Daniel · Lucas · Perla · Rildo

Junho de 2026

Sumário

1. Introdução

Este documento apresenta, de forma detalhada, o estudo desenvolvido pela Equipe 3 sobre a estrutura de dados conhecida como Árvore Rubro-Negra (Red-Black Tree), no âmbito da disciplina de Estruturas de Dados II. O trabalho foi estruturado em quatro aulas expositivas e práticas, abordando desde os fundamentos teóricos da estrutura até sua implementação em código e a resolução de exercícios práticos.

As Árvores Rubro-Negras são uma das estruturas de dados auto-balanceadas mais utilizadas na prática da computação, estando presentes em bibliotecas padrão de linguagens como C++ (`std::map`, `std::set`) e Java (`TreeMap`, `TreeSet`), além de compor estruturas internas do núcleo do sistema operacional Linux. Seu estudo é fundamental para a compreensão de como sistemas reais garantem operações eficientes de busca, inserção e remoção em grandes volumes de dados.

O objetivo deste trabalho é apresentar de forma clara e progressiva: (i) a definição formal e as propriedades que caracterizam uma Árvore Rubro-Negra; (ii) os mecanismos de rotação e recoloração que garantem o balanceamento da estrutura; (iii) os algoritmos de inserção e remoção, incluindo o tratamento de violações; (iv) a análise de complexidade que justifica sua ampla utilização; (v) uma implementação prática em código; e (vi) exercícios resolvidos que consolidam o aprendizado teórico.

2. Divisão do Trabalho e Responsabilidades

Conforme orientação da disciplina, o trabalho foi dividido em quatro aulas, cada uma com foco específico, e cada integrante da equipe assumiu um papel definido nas etapas de pesquisa, desenvolvimento e apresentação. A tabela a seguir resume a divisão de papéis e responsabilidades dentro da equipe.

Integrante	Papel	Responsabilidade Principal
Daniel	Coordenador	Coordenação geral do projeto, abertura da apresentação e introdução ao tema (Aula 1).
Lucas	Especialista em Fundamentos	Pesquisa teórica sobre a estrutura, histórico e propriedades fundamentais da árvore rubro-negra (Aula 1).
Chris	Analista de Operações	Mapeamento visual das rotações e demonstrações gráficas do mecanismo de balanceamento (Aula 2).
Rildo	Engenheiro de Desempenho	Algoritmos de inserção e remoção, prova de complexidade $O(\log n)$ e análise de casos (Aula 2).

Integrante	Papel	Responsabilidade Principal
Adilson	Arquiteto de Software	Implementação e validação do código-fonte da estrutura, disponibilizado em repositório público (Aula 3).
Perla	Designer de Exercícios / Mediadora	Elaboração dos exercícios práticos, quiz interativo de fixação e condução da aula prática (Aula 4).

3. Fundamentos Teóricos

3.1. Definição

Uma Árvore Rubro-Negra é uma árvore binária de busca (ABB) auto-balanceada na qual cada nó armazena, além da chave e dos ponteiros para os filhos, um atributo de cor: vermelho ou preto. Essa coloração não é arbitrária — ela é mantida segundo um conjunto rígido de regras que, juntas, garantem que a altura da árvore permaneça proporcional a $O(\log n)$, mesmo após sucessivas inserções e remoções.

A estrutura foi originalmente proposta por Rudolf Bayer em 1972, sob o nome de “B-trees simétricas”, e posteriormente formalizada com a nomenclatura de cores por Leonidas J. Guibas e Robert Sedgwick em 1978, em seu artigo “A Dichromatic Framework for Balanced Trees”.

3.2. As Cinco Propriedades Fundamentais

Para que uma árvore binária de busca seja classificada como uma Árvore Rubro-Negra válida, ela deve satisfazer simultaneamente as cinco propriedades a seguir:

1. Todo nó da árvore é VERMELHO ou PRETO.
2. A raiz da árvore é sempre PRETA.
3. Toda folha (representada pelo nó sentinela NIL) é considerada PRETA.
4. Se um nó é VERMELHO, então ambos os seus filhos são PRETOS — ou seja, não pode haver dois nós vermelhos consecutivos em nenhum caminho da árvore.
5. Para cada nó, todos os caminhos simples daquele nó até as folhas descendentes contêm o mesmo número de nós pretos. Esse número é denominado altura preta (black-height) do nó.

Consequência direta: a combinação dessas cinco propriedades garante que o caminho mais longo da raiz até uma folha nunca seja mais do que o dobro do caminho mais curto, o que limita a altura da árvore a, no máximo, $2 \cdot \log_2(n + 1)$, sendo n o número de nós internos.

3.3. Altura Preta (Black-Height)

A altura preta de um nó x , denotada $bh(x)$, é definida como o número de nós pretos em qualquer caminho simples de x (não incluindo x) até uma folha descendente, sem contar a própria folha NIL. Pela Propriedade 5, esse valor é necessariamente o mesmo para todos os caminhos a partir de um mesmo nó, o que torna o conceito bem definido.

O lema fundamental que sustenta a eficiência das Árvores Rubro-Negras pode ser enunciado da seguinte forma: uma árvore rubro-negra com n nós internos possui altura h tal que $h \leq 2 \cdot \log_2(n + 1)$. Esse resultado é o que assegura que as operações de busca, inserção e remoção sejam executadas em tempo $O(\log n)$ no pior caso.

3.4. Comparação com Outras Estruturas Balanceadas

É importante posicionar a Árvore Rubro-Negra em relação a outras estruturas de dados com objetivos semelhantes, de modo a justificar sua ampla adoção prática.

Estrutura	Característica de Balanceamento	Complexidade no Pior Caso
ABB comum	Nenhum balanceamento automático.	$O(n)$
Árvore AVL	Balanceamento estrito (diferença de altura ≤ 1); rotações mais frequentes.	$O(\log n)$
Rubro-Negra	Balanceamento relaxado via cores; rotações menos frequentes em média.	$O(\log n)$

Em termos práticos, a Árvore AVL oferece buscas ligeiramente mais rápidas por ser mais rigidamente balanceada, porém a Árvore Rubro-Negra realiza, em média, menos rotações durante inserções e remoções, o que a torna mais eficiente em cenários com alta frequência de modificações — razão pela qual é a escolha preferencial em bibliotecas padrão de linguagens de programação.

3.5. Aplicações Práticas

As Árvores Rubro-Negras não são apenas um exercício teórico: elas estão presentes em diversos sistemas amplamente utilizados na indústria de software, conforme listado a seguir.

- Sistemas Operacionais: o kernel do Linux utiliza Árvores Rubro-Negras para gerenciar o escalonador de processos Completely Fair Scheduler (CFS) e estruturas de memória virtual.
- Linguagens de Programação: a biblioteca padrão do C++ implementa `std::map` e `std::set` internamente como Árvores Rubro-Negras; o Java faz o mesmo com `TreeMap` e `TreeSet`.
- Bancos de Dados: índices mantidos em memória utilizam estruturas balanceadas como esta para garantir buscas em tempo logarítmico.
- Roteadores de Rede: tabelas de roteamento IP utilizam estruturas balanceadas para inserção e consulta eficientes de rotas.

4. Rotações: o Mecanismo de Balanceamento

As rotações constituem o mecanismo central pelo qual uma Árvore Rubro-Negra restaura suas propriedades estruturais após uma inserção ou remoção. Trata-se de operações locais, executadas em tempo constante $O(1)$, que reorganizam um pequeno conjunto de ponteiros sem violar a propriedade de ordenação da árvore binária de busca.

4.1. Rotação à Esquerda (Left-Rotate)

Na rotação à esquerda aplicada sobre um nó x , o filho direito de x , denominado y , assume a posição de x na árvore. O próprio nó x passa a ser o filho esquerdo de y , e a subárvore que antes era filha esquerda de y passa a ser a subárvore filha direita de x . Essa operação preserva integralmente a ordenação dos elementos.

4.2. Rotação à Direita (Right-Rotate)

A rotação à direita é simétrica à rotação à esquerda: o filho esquerdo de um nó y , denominado x , assume a posição de y , que passa a ser o filho direito de x . A subárvore que era filha direita de x passa a ser filha esquerda de y .

Ambas as rotações são operações $O(1)$: apenas um número fixo de ponteiros é alterado, independentemente do tamanho da árvore. A complexidade total das operações de inserção e remoção é dominada pela busca da posição correta ($O(\log n)$) e não pelas rotações em si.

5. Algoritmo de Inserção

O processo de inserção em uma Árvore Rubro-Negra ocorre em duas etapas distintas: primeiro, o novo nó é inserido seguindo exatamente o procedimento padrão de uma árvore binária de

busca; em seguida, é executado um procedimento de correção (fixup) para restaurar eventuais violações das propriedades da estrutura.

5.1. Regra Inicial

Todo nó recém-inserido recebe, por padrão, a cor VERMELHA. Essa escolha é deliberada: inserir um nó preto alteraria imediatamente a altura preta de um dos caminhos da árvore, violando a Propriedade 5 de forma mais grave e custosa de corrigir. Ao inserir em vermelho, a única violação possível é a da Propriedade 4 (dois vermelhos consecutivos).

5.2. O Conflito e o Papel do Tio

Se o nó pai do elemento recém-inserido também for vermelho, ocorre uma violação da Propriedade 4. Para corrigir essa violação, o algoritmo analisa a cor do “tio” do novo nó — isto é, o irmão do seu nó pai — e decide entre três casos de correção, descritos na tabela a seguir.

Caso	Condição	Ação de Correção
Caso 1	O tio do novo nó é VERMELHO.	Recolorir: pai e tio tornam-se PRETOS; o avô torna-se VERMELHO. O problema é deslocado para o nível do avô (recursão).
Caso 2	O tio é PRETO e o novo nó é filho à DIREITA do pai.	Executa-se uma rotação à esquerda sobre o pai, transformando a configuração no Caso 3.
Caso 3	O tio é PRETO e o novo nó é filho à ESQUERDA do pai.	Recolorem-se pai e avô; executa-se uma rotação à direita sobre o avô.

Ao final do procedimento de correção, é necessário garantir explicitamente que a raiz da árvore permaneça PRETA, conforme exige a Propriedade 2 — esta verificação é sempre a última etapa do algoritmo, independentemente de qual caso tenha sido executado.

5.3. Exemplo Resolvido

Considere a árvore inicial composta pela raiz 20 (preta), com filho esquerdo 10 (vermelho) e filho direito 30 (preto). Deseja-se inserir os valores 25 e, em seguida, 5.

- Inserção do 25: como $25 > 20$, o algoritmo segue para a direita; como $25 < 30$, o novo nó torna-se filho esquerdo de 30. Como o pai (30) é preto, não há violação.
- Inserção do 5: como $5 < 20$, segue-se para a esquerda; como $5 < 10$, o novo nó torna-se filho esquerdo de 10. Como o pai (10) é vermelho, ocorre uma violação da Propriedade 4.
- Diagnóstico: a configuração $20 \rightarrow 10 \rightarrow 5$ caracteriza o padrão “LL” (esquerda-esquerda), correspondendo ao Caso 3.

- Correção: executa-se uma rotação à direita sobre o nó 20 — o nó 10 assume a posição de raiz da subárvore e o nó 20 desce para a direita. Em seguida, recolorem-se os nós: 10 passa a PRETO e 20 passa a VERMELHO.

O resultado final é uma árvore com raiz 10 (preta), filho esquerdo 5 (vermelho) e filho direito 20 (vermelho), sendo que 20 possui, por sua vez, os filhos 25 (preto) e 30 (preto). Todas as cinco propriedades permanecem satisfeitas.

6. Algoritmo de Remoção

A remoção é reconhecidamente a operação mais complexa sobre Árvores Rubro-Negras, pois a retirada de um nó pode reduzir a altura preta de um dos caminhos da árvore, violando a Propriedade 5. O algoritmo de remoção é estruturado em três etapas principais.

6. Remoção padrão de ABB: o nó é removido seguindo o procedimento clássico de árvores binárias de busca. Caso o nó possua dois filhos, ele é substituído pelo seu sucessor in-order (o menor elemento da subárvore direita).
7. Verificação de violação: caso o nó efetivamente removido da estrutura fosse PRETO, a contagem de nós pretos em um dos caminhos é reduzida, dando origem ao conceito de nó “duplo-preto” — um artifício temporário utilizado para preservar a contagem durante a correção.
8. Correção (RB-Delete-Fixup): o algoritmo analisa a cor do “irmão” do nó duplo-preto e de seus sobrinhos para decidir entre rotações e recolorações que restaurem o equilíbrio da árvore.

Conceito-chave: remover um nó VERMELHO nunca quebra a Propriedade 5, pois nós vermelhos não são contados na altura preta. A complexidade da remoção surge exclusivamente quando o nó removido (ou seu substituto efetivo) é PRETO.

6.1. Exemplo Resolvido

Considere a árvore inicial com raiz 50 (preta), filhos 30 (vermelho) e 70 (vermelho), sendo que 30 possui os filhos 20 e 40 (ambos pretos) e 70 possui o filho 80 (preto). Deseja-se remover, nessa ordem, os valores 30 e 70.

- Remoção do 30: como o nó possui dois filhos, ele é substituído pelo seu sucessor in-order, o valor 40, que assume a posição de 30 na árvore. Como o nó removido (30) era vermelho, nenhuma violação de altura preta ocorre.
- Remoção do 70: o nó 70 é vermelho e possui apenas um filho (80, preto). Trata-se, portanto, de uma substituição simples — o nó 80 assume diretamente a posição de 70, sem necessidade de recoloração ou rotação.

O resultado final é uma árvore com raiz 50 (preta), filho esquerdo 40 (vermelho, com filho preto) e filho direito 80 (preto), preservando integralmente as cinco propriedades da estrutura.

7. Análise de Complexidade

A principal justificativa para a adoção das Árvores Rubro-Negras em sistemas de produção é a garantia de complexidade $O(\log n)$ em todas as operações fundamentais, mesmo no pior caso — diferentemente de uma árvore binária de busca comum, cuja complexidade pode degenerar para $O(n)$ caso a árvore não seja balanceada.

Operação	Melhor Caso	Pior Caso	Rotações
Busca	$O(\log n)$	$O(\log n)$	$O(1)$
Inserção	$O(\log n)$	$O(\log n)$	$O(\log n)$
Remoção	$O(\log n)$	$O(\log n)$	$O(\log n)$
Mínimo / Máximo	$O(\log n)$	$O(\log n)$	—
Sucessor	$O(\log n)$	$O(\log n)$	—

7.1. Por que $O(\log n)$? Uma Explicação Intuitiva

O significado prático da notação $O(\log n)$ é que o tempo de execução das operações cresce de forma extremamente lenta em relação ao número de elementos armazenados. Isso ocorre porque, a cada decisão tomada durante a busca pela posição correta (seguir para a esquerda ou para a direita), o algoritmo elimina, em média, metade dos elementos restantes a serem considerados.

Para ilustrar essa propriedade de forma concreta, considere os exemplos numéricos a seguir:

- Uma árvore com 1.000 elementos requer, no máximo, aproximadamente 10 comparações para localizar qualquer elemento.
- Uma árvore com 1.000.000 de elementos requer, no máximo, aproximadamente 20 comparações — o dobro do esforço para mil vezes mais dados.

Essa característica é o que torna as Árvores Rubro-Negras adequadas para sistemas que precisam lidar com grandes volumes de dados sem degradação perceptível de desempenho.

7.2. Comparação com a Árvore AVL

Embora ambas as estruturas garantam complexidade $O(\log n)$, existem diferenças práticas relevantes. A Árvore AVL, por impor um critério de balanceamento mais rígido (a diferença de altura entre subárvores nunca pode exceder 1), tende a apresentar buscas ligeiramente mais

rápidas. Em contrapartida, a Árvore Rubro-Negra, por adotar um critério de balanceamento mais flexível baseado em cores, realiza, em média, um número menor de rotações durante inserções e remoções, o que a torna preferível em aplicações com alta frequência de modificações na estrutura — como é o caso de bibliotecas padrão de linguagens de programação.

8. Implementação

Como parte prática do trabalho, foi desenvolvida uma implementação completa da Árvore Rubro-Negra na linguagem Python, contemplando as operações de inserção, remoção, busca e os respectivos procedimentos de correção (fixup). O código foi organizado de forma comentada e didática, com o objetivo de servir como material de apoio ao aprendizado da turma.

8.1. Estrutura do Nó

Cada nó da árvore é representado por uma classe contendo a chave armazenada, a cor (vermelho ou preto), e referências para o filho esquerdo, o filho direito e o nó pai. Por padrão, todo nó recém-criado é inicializado com a cor vermelha, conforme discutido na Seção 5.1.

```
RED, BLACK = 0, 1

class Node:
    def __init__(self, key):
        self.key    = key
        self.color  = RED      # novo nó nasce vermelho
        self.left   = None
        self.right  = None
        self.parent = None
```

8.2. Rotação à Esquerda

A função a seguir implementa a rotação à esquerda descrita na Seção 4.1, operando exclusivamente sobre ponteiros locais, o que garante sua complexidade $O(1)$.

```
def left_rotate(self, x):
    y = x.right
    x.right = y.left
    if y.left is not None:
        y.left.parent = x
    y.parent = x.parent
    if x.parent is None:
```

```
        self.root = y
    elif x == x.parent.left:
        x.parent.left = y
    else:
        x.parent.right = y
    y.left = x
    x.parent = y
```

8.3. Inserção e Correção

A função de inserção segue o procedimento padrão de árvores binárias de busca para localizar a posição do novo nó, e em seguida invoca a rotina de correção (fixup), que implementa os três casos apresentados na Seção 5.2.

```
def insert(self, key):
    z = Node(key)
    y, x = None, self.root
    while x:
        y = x
        x = x.left if z.key < x.key else x.right
    z.parent = y
    if y is None:
        self.root = z
    elif z.key < y.key:
        y.left = z
    else:
        y.right = z
    self._insert_fixup(z)
```

8.4. Repositório de Código

A implementação completa, incluindo a remoção, os casos de teste e a documentação de uso, está disponível publicamente no seguinte repositório:

Repositório GitHub: github.com/JAdilsonSA/rvore-RN

9. Exercícios Práticos Resolvidos

Com o objetivo de consolidar a compreensão teórica da turma, foram elaborados três exercícios práticos, cada um focado em uma operação fundamental da estrutura: inserção, rotação e remoção. Os exercícios e suas respectivas resoluções são apresentados a seguir.

9.1. Exercício 1 — Inserção

Enunciado: a partir da árvore inicial composta pela raiz 20 (preta), filho esquerdo 10 (vermelho) e filho direito 30 (preto), inserir os valores 25 e 5, nessa ordem, seguindo as regras da árvore rubro-negra, corrigindo eventuais violações e apresentando a árvore final balanceada.

Resolução: a inserção do valor 25 ocorre sem violações, pois seu nó pai (30) é preto. A inserção do valor 5, entretanto, resulta em uma violação da Propriedade 4, uma vez que seu pai (10) também é vermelho. Identificada a configuração LL ($20 \rightarrow 10 \rightarrow 5$), aplica-se uma rotação à direita sobre o nó 20, seguida da recoloração do nó 10 para preto e do nó 20 para vermelho. A árvore final resultante possui raiz 10 (preta), com filhos 5 (vermelho) e 20 (vermelho), sendo que este último possui os filhos 25 e 30, ambos pretos.

9.2. Exercício 2 — Rotação

Enunciado: dada a árvore com raiz 30 (preta), filhos 20 (vermelho) e 40 (preto), sendo que 20 possui o filho 10 (vermelho), identificar o problema de balanceamento existente, executar a rotação necessária e corrigir as cores.

Resolução: identifica-se uma violação da Propriedade 4 entre os nós 20 e 10 (ambos vermelhos), em uma configuração LL ($30 \rightarrow 20 \rightarrow 10$). A correção consiste em uma rotação à direita sobre o nó 30, fazendo com que o nó 20 assuma a posição de raiz da subárvore. Em seguida, recolorem-se os nós: 20 passa a PRETO e 30 passa a VERMELHO. O resultado final apresenta o nó 20 como raiz (preto), com filhos 10 (vermelho) e 30 (vermelho), sendo que este último possui o filho 40 (preto).

9.3. Exercício 3 — Remoção

Enunciado: a partir da árvore com raiz 50 (preta), filhos 30 (vermelho) e 70 (vermelho) — sendo que 30 possui os filhos 20 e 40 (pretos) e 70 possui o filho 80 (preto) — remover os valores 30 e 70, nessa ordem, respeitando as regras da árvore rubro-negra e mantendo suas propriedades.

Resolução: a remoção do valor 30 é resolvida pela substituição do nó por seu sucessor in-order, o valor 40, sem geração de violações, pois o nó removido era vermelho. A remoção subsequente do valor 70 — também vermelho, com um único filho preto (80) — é resolvida por substituição simples, sem necessidade de rotações ou recolorações. A árvore final resultante apresenta raiz 50 (preta), filho esquerdo 40 (vermelho, com filho 20 preto) e filho direito 80 (preto).

9.4. Quiz Interativo de Fixação

Como atividade complementar de fixação, foi elaborado um quiz interativo, disponibilizado através da plataforma Wayground, abordando os temas de propriedades estruturais, casos de inserção, tipos de rotação, complexidade assintótica e aplicações práticas da estrutura estudada.

Quiz: wayground.com/admin/quiz/6a2ebd7486925a22f2ecd4a0

10. Cronograma de Apresentação

Conforme estabelecido pela proposta da disciplina, a apresentação do seminário foi estruturada em quatro aulas, seguindo uma progressão lógica que parte do conceito abstrato até a implementação e o uso prático da estrutura.

Aula	Foco	Conteúdo Abordado
Aula 1	Teoria: Fundamentos	Definição da estrutura, histórico, propriedades matemáticas e regras básicas de coloração. Apresentado por Daniel e Lucas.
Aula 2	Teoria: Mecânica e Complexidade	Rotações, algoritmos de inserção e remoção, análise de complexidade temporal $O(\log n)$ e aplicações no mundo real. Apresentado por Chris e Rildo.
Aula 3	Prática: Implementação	Demonstração guiada do código (live coding / code walkthrough), com construção da estrutura passo a passo. Apresentado por Rildo e Adilson.
Aula 4	Prática: Hands-on	Resolução de exercícios pela turma, quiz interativo de fixação e encerramento das atividades. Apresentado por Perla.

11. Conclusão

O estudo das Árvores Rubro-Negras permitiu à equipe compreender, de forma aprofundada, como estruturas de dados auto-balanceadas conseguem garantir desempenho previsível mesmo diante de operações sucessivas de inserção e remoção. A combinação de regras de coloração relativamente simples — apenas cinco propriedades — é suficiente para assegurar, matematicamente, que a altura da árvore nunca exceda $2 \cdot \log_2(n + 1)$, garantindo complexidade $O(\log n)$ em todas as operações fundamentais.

A relevância prática da estrutura foi evidenciada por sua presença em sistemas amplamente utilizados, como o núcleo do sistema operacional Linux e as bibliotecas padrão das linguagens C++ e Java, o que reforça a importância de seu estudo para a formação de profissionais de computação.

A divisão do trabalho em papéis específicos — coordenação, pesquisa teórica, análise de operações, engenharia de desempenho, arquitetura de software e elaboração de exercícios — permitiu que cada integrante da equipe desenvolvesse competências complementares, desde a fundamentação matemática até a implementação prática em código, consolidando o aprendizado de forma colaborativa e multidisciplinar.

12. Referências Bibliográficas

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 4. ed. Cambridge: MIT Press, 2022. Capítulo 13: Red-Black Trees.

SEEDGEWICK, Robert; WAYNE, Kevin. Algorithms. 4. ed. Boston: Addison-Wesley, 2011. Capítulo 3.3: Balanced Search Trees.

GUIBAS, Leonidas J.; SEEDGEWICK, Robert. A Dichromatic Framework for Balanced Trees. In: Proceedings of the 19th Annual Symposium on Foundations of Computer Science (FOCS). 1978. p. 8–21.

BAYER, Rudolf. Symmetric Binary B-Trees: Data Structure and Maintenance Algorithms. Acta Informatica, v. 1, n. 4, p. 290–306, 1972.

CPPREFERENCE. `std::map` — C++ Standard Library Reference. Disponível em: <https://en.cppreference.com/w/cpp/container/map>. Acesso em: jun. 2026.

Equipe 3 — Estruturas de Dados II